NTNU

TDT4225 – Very Large, Distributed Data Volumes

Oppgave 3

Utforskende dataanalyse og spørringer i MongoDB

Gruppe 78

Didrik Røssum Jensen

Muhammed Tahir Adil

Oktober 2026

Innholdsfortegnelse

1 Innledning **2**

2 Resultater **2**

2.1 Del 1: EDA og klargjøring av data . . . 2

2.1.1 Datasett og datakvalitet . . . 2

2.1.2 Datavask . . . 6

2.1.3 Databasemodell og innsetting . . . 7

2.2 Del 2: Spørringer . . . 8

2.2.1 Oppgave 1: Regissører med høyest medianinntekt . . . 8

2.2.2 Oppgave 2: Skuespillerpar . . . 10

2.2.3 Oppgave 3: Skuespillere med størst sjangerbredde . . . 12

2.2.4 Oppgave 4: Filmsamlinger med størst samlet inntekt . . . 14

2.2.5 Oppgave 5: Spilletid per tiår og primærsjanger . . . 15

2.2.6 Oppgave 6: Kvinneandel blant de fem øverst krediterte . . . 17

2.2.7 Oppgave 7: Tekstsøk etter noir og neo-noir . . . 19

2.2.8 Oppgave 8: Regissør–skuespiller-samarbeid . . . 20

2.2.9 Oppgave 9: Ikke-engelske filmer med amerikansk tilknytning . . . 21

2.2.10 Oppgave 10: Brukernes sjangerbredde og vurderingsvarians . . . 23

3 Diskusjon **24**

3.1 EDA og manglende verdier . . . 24

3.2 Datamodell . . . 25

3.3 Datatyper . . . 25

3.4 Ytelse og begrensninger . . . 25

4 Hva lærte vi **26**

5 Tilbakemelding **26**

6 Bruk av KI **26**

Referanser **27**

1 Innledning

Oppgaven går ut på å utforske, rense og behandle «The Movies Dataset» fra Kaggle (Banik, u.å.). Datasettet inneholder informasjon om over 45 000 filmer, inkludert metadata, skuespillere, filmcrew og nøkkelord, samt rundt 26 millioner brukervurderinger. Etter databehandlingen opprettes kolleksjoner i MongoDB, og dataene settes inn før spørringer utføres for å besvare spørsmålene i oppgaven.

Gruppen består av Tahir og Didrik, som har samarbeidet om å løse oppgaven. Arbeidet er fordelt mellom gruppemedlemmene med ulike ansvarsområder innen datautforskning og databehandling. Samarbeidet om koden foregår gjennom versjonskontroll med Git og GitHub. Repositoriet finnes på GitHub.

Målet er å organisere strukturerte og delvis strukturerte filmdata i en dokumentdatabase, slik at spørsmål om blant annet regissører, skuespillere, sjangre, inntekter og brukervurderinger kan besvares med MongoDB-spørringer. Metadata, cast, crew og keywords samles i filmdokumenter, mens brukervurderinger lagres i en egen kolleksjon med referanser til filmene.

Verktøyene som benyttes, er:

- Pandas til databehandling og utforskning.
- Matplotlib til visualisering.
- PyMongo til kommunikasjon mellom Python og MongoDB.
- Jupyter til EDA.
- Docker til lokal kjøring av MongoDB (versjon 7.0.1).
- Git og GitHub til versjonskontroll og samarbeid.

2 Resultater

2.1 Del 1: EDA og klargjøring av data

2.1.1 Datasett og datakvalitet

Vi bruker fem filer fra datasettet: `movies_metadata.csv` (45 466 rader, 24 kolonner), `credits.csv` (cast og crew per film), `keywords.csv`, `links.csv` (45 843 rader) og `ratings.csv` (26 024 289 ratings fra 270 896 brukere). Filene kobles på to typer ID-er: `movies_metadata`, `credits` og `keywords` bruker TMDB-ID, mens `ratings` bruker MovieLens-ID. `links.csv` oversetter mellom de to. De mindre filene (`ratings_small` og `links_small`) brukes bare til testing, siden `ratings_small` ikke er et utsnitt av `ratings` (bare 6 % av bruker–film-parene går igjen).

Nestede felt som `genres`, `cast` og `crew` er lagret som tekst i CSV-filene. Teksten er Python-repr og ikke JSON, så den må leses med `ast.literal_eval`. `budget`, `id` og `popularity` er lest inn som tekst fordi tre rader er ødelagt.

Fil	Innhold (feltene vi bruker)	Nøkkel
`movies_metadata`	tittel, `release_date`, `runtime`, `budget`, `revenue`, `vote_average`, `vote_count`, `original_language`, `overview`, `tagline`, `genres`, `production_companies`, `production_countries`, `belongs_to_collection`	`id` (TMDB)
`credits`	`cast` (navn, `order`, `gender`) og `crew` (navn, `job`, `department`)	`id` (TMDB)
`keywords`	`keywords` (id og navn)	`id` (TMDB)
`links`	`movieId`, `imdbId`, `tmdbId`	`movieId`
`ratings`	`userId`, `movieId`, `rating` (0,5–5), `timestamp`	`(userId, movieId)`

Tabell 1: Filene i datasettet, feltene som brukes i oppgavene og kolonnene filene kobles på.

Hovedfunn om datakvaliteten:

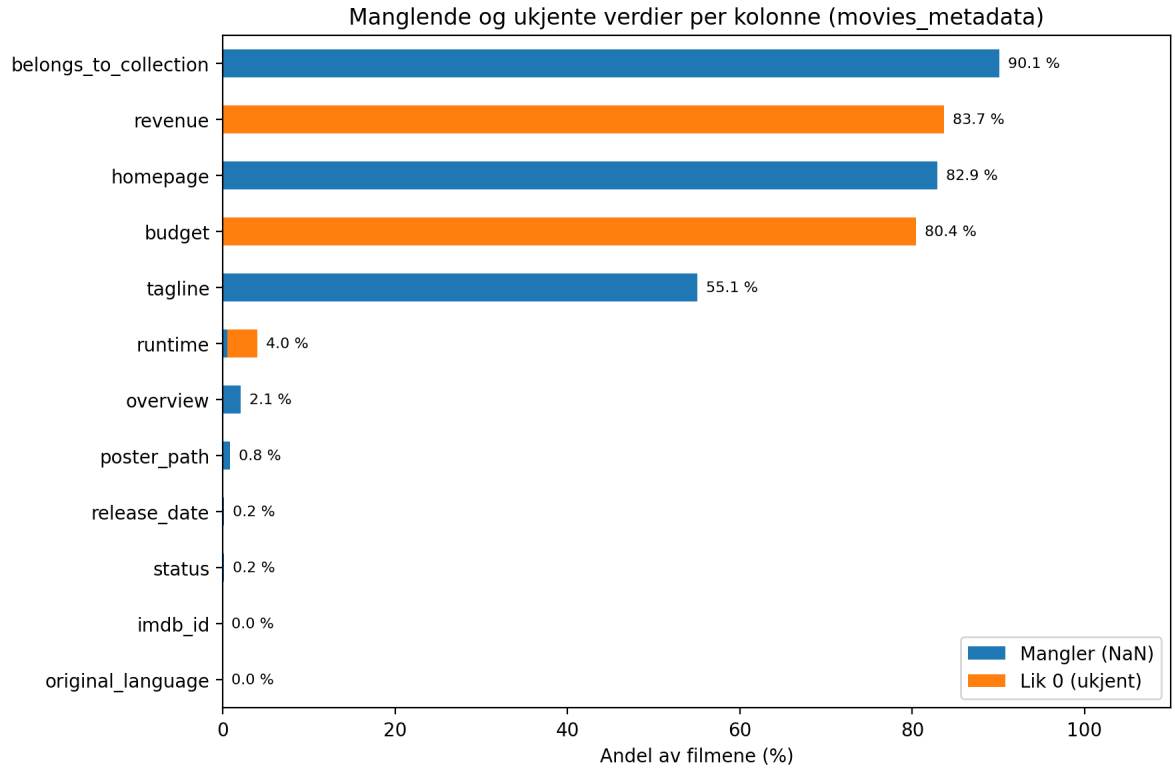
- **Ødelagte rader.** Tre filmer er splittet over to CSV-rader hver (6 rader med 10 og 15 felt i stedet for 24), fordi `overview` inneholder linjeskift. Det gir tre rader med ugyldig `id`.
- **Duplikater.** I `movies_metadata` er 17 rader helt like, og 13 `id`-er har to rader med ulik `popularity`. I `credits` finnes 37 helt like rader og 7 ekstra `id`-forekomster. I `keywords` er 987 rader helt like.
- **Skjulte manglende verdier.** `budget` er 0 i 36 557 rader (ca. 80 %), `revenue` er 0 i 38 041 (ca. 84 %), og `runtime` er 0 i 1 558. Vi behandler 0 som «ukjent» i disse feltene; datasettet alene bekrefter ikke at samtlige nuller er uregistrerte verdier. I `credits` har 223 765 cast-oppføringer `gender` 0 (ukjent kjønn).
- **Ekte manglende verdier.** `release_date` mangler for 84 filmer, `original_language` for 11 og `status` for 81. 946 filmer mangler både `overview` og `tagline`, 2 414 har tom cast og 887 mangler regissør.
- **Koblingen mellom ID-ene.** `tmdbId` mangler for 219 av 45 843 rader i `links`, og 161 `tmdbId` finnes ikke i `movies_metadata`. 29 `tmdbId` deles av flere `movieId` (maks 3). Likevel kan 99,84 % av ratingene kobles til en film.
- **Skjeve fordelinger.** Gjennomsnittsbudsjettet (for filmer med budsjett over 0) er ca. 21,6 mill. mot en median på 8 mill., og median `vote_count` er bare 10. Derfor er median et bedre mål enn gjennomsnitt i oppgavene, og `vote_average` er upålitelig for filmer med få stemmer.

Vi imputerte ikke manglende beløp. Når rundt 80 % av budsjettene og 84 % av inntektene er 0, ville innfylling med et standardbeløp tilført antatte verdier til analysene. Vi beholder filmene, men lar bare registrerte positive inntekter inngå i inntektsmedianer og oppgir datagrunnlaget separat.

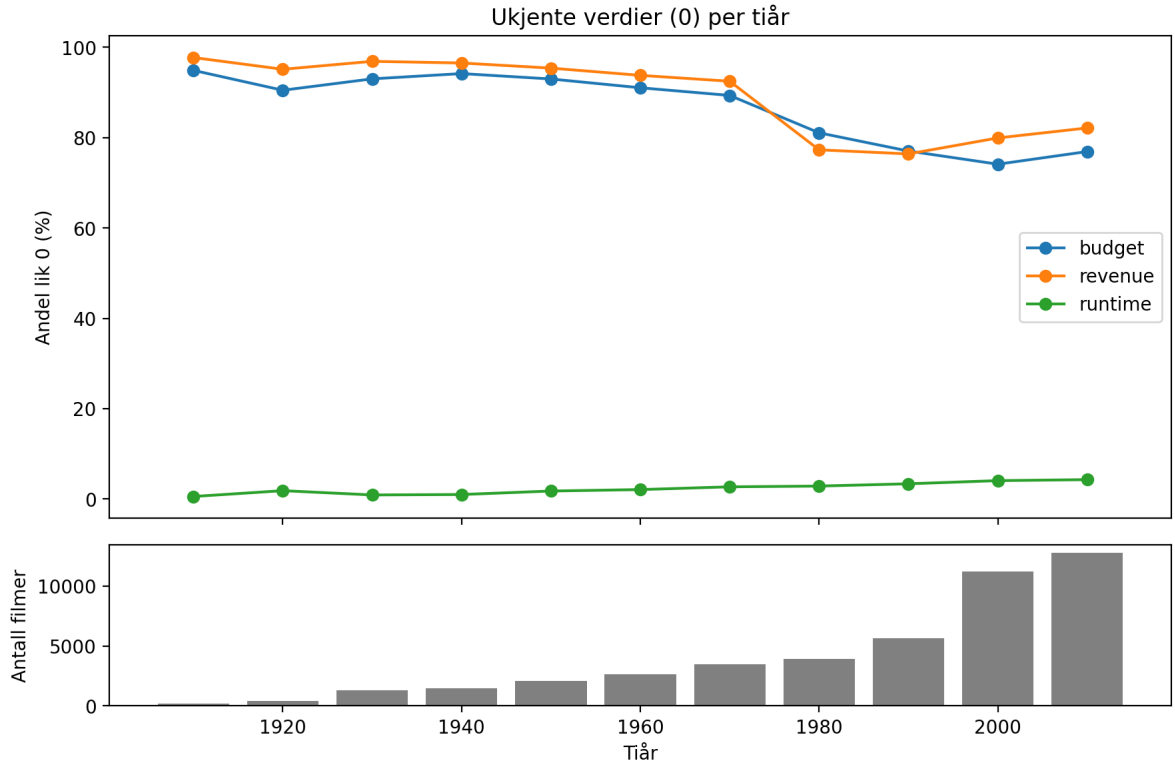
Kolonne	Antall NaN	Andel
belongs_to_collection	40 972	90.1 %
homepage	37 684	82.9 %
tagline	25 050	55.1 %
overview	954	2.1 %
poster_path	383	0.8 %
runtime	257	0.6 %
release_date	84	0.2 %
status	81	0.2 %
imdb_id	17	0.0 %
original_language	11	0.0 %

Figur 1: Antall og andel manglende (NaN) verdier per kolonne i `movies_metadata`, etter at de tre ødelagte radene er reparert.

Figur 2 viser omfanget av manglende og ukjente verdier i `movies_metadata`. Mangelen på `budget` og `revenue` er ikke bare et problem for gamle filmer. Figur 3 viser at andelen ukjente verdier er rundt 90–97 % frem til 1970-tallet og faller til 74–82 % fra 1980-tallet, men den er fortsatt høy i alle tiår.



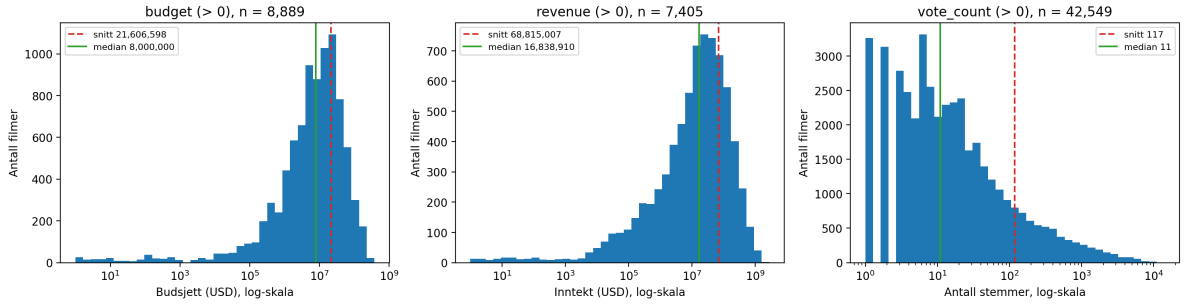
Figur 2: Andel manglende (NaN) og ukjente (0) verdier per kolonne i `movies_metadata`.



Figur 3: Andel av `budget`, `revenue` og `runtime` som er 0, per tiår (bare tiår med minst 100 filmer), med antall filmer per tiår under.

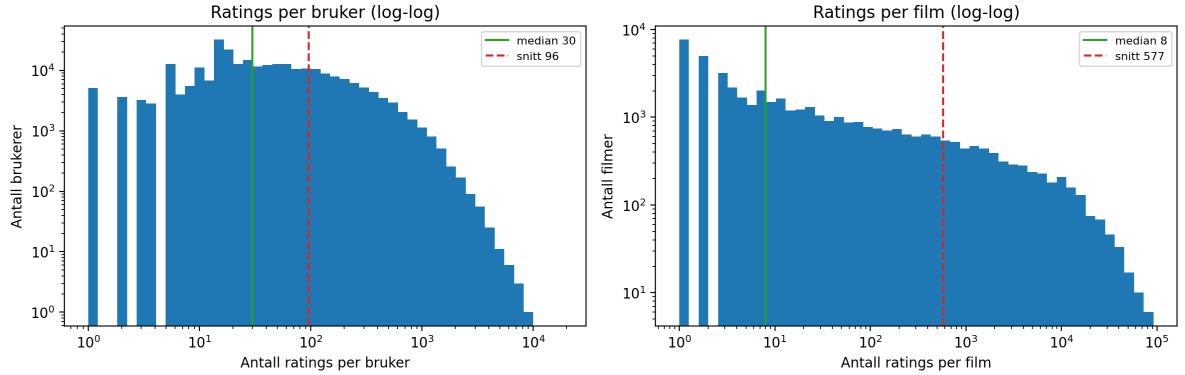
Figur 4 viser fordelingen av `budget`, `revenue` og `vote_count` på logaritmisk akse. Alle tre er

sterkt høyreskjeve, og snittet ligger langt over medianen. Det begrunner at oppgavene bruker median. Figuren viser også en hale med budsjetter under 1 000 USD. Datasettet gir ikke grunnlag for å avgjøre om disse skyldes lave budsjetter, feil enhet eller andre registreringsfeil. Medianen av `vote_count` er 11 her fordi figuren bare teller filmer med minst én stemme (10 når alle filmer teller med).



Figur 4: Fordeling av `budget`, `revenue` og `vote_count` (verdier over 0, logaritmisk x-akse). Rød stiplet linje er snitt og grønn linje er median.

Antall ratings er også svært skjevt fordelt (figur 5): median 30 ratings per bruker mot et snitt på 96, og median 8 per film mot et snitt på 577. Noen få brukere og filmer står for svært mange ratings, noe som er grunnen til at ratings ligger i en egen samling og at oppgave 10 må lese alle 26 millioner dokumentene.



Figur 5: Antall ratings per bruker og per film i `ratings.csv` (logaritmiske akser).

2.1.2 Datavask

Råfilene endres aldri. All rensing skjer i Python i `insert_data.py` før innsetting. Etter rensing står vi igjen med 45 433 filmer (fra 45 466 rader) og 26 024 289 ratings (ingen er fjernet).

movies_metadata.

- De tre splittede filmene settes sammen igjen (fortsettelsen av `overview` legges til teksten, og de siste feltene flyttes til riktige kolonner). Det gir 45 463 rader.
- `id`, `budget` og `popularity` gjøres om til tall og `release_date` til `datetime`. Ugyldige verdier gir feilmelding i stedet for å bli `NaN` i det stille.
- De 17 helt like radene fjernes. For de 13 `id`-ene med to ulike rader beholdes raden med høyest `vote_count`, og `popularity` settes til snittet av de to. Det gir én rad per `id` (45 433 filmer), som trengs fordi `id` blir `_id`.

- De nestede liste- og objektfeltene parses med `ast.literal_eval`. `belongs_to_collection` lagres som `null` når en samling mangler.
- Manglende verdier imputeres ikke. Manglende `release_date`, `original_language` og `status` lagres som `null`. Verdiene 0 i `budget`, `revenue` og `runtime` lagres uendret, og spørringene må selv hoppe over dem.
- Lange `runtime`-verdier (for eksempel 1 256 minutter) beholdes, siden de ikke er bevist feil. Vi bruker median, som tåler slike verdier.

credits og keywords.

- Helt like rader fjernes. De 7 gjenværende `id`-parene i `credits` har samme innhold i ulik rekkefølge, så vi beholder første rad per `id`.
- `cast`, `crew` og `keywords` parses med `ast.literal_eval` og slås sammen med filmene på `id` (en-til-en). Filmer uten treff får tom liste.
- 4 keyword-oppføringer som bare har mellomrom som navn (id 13 091) fjernes.

links og ratings.

- `links` leses inn og `tmdbId` (som er `float`) gjøres om til heltall. En referanse settes bare hvis filmen finnes i `movies`, ellers `null`.
- Ratings uten treff beholdes (42 707 stk.), med `tmdbId = null`. Antall ratings og varians per bruker er fortsatt gyldige, og bare sjangertellingen i oppgave 10 trenger koblingen.
- De 29 `tmdbId`-ene som deles av flere `movieId` påvirker 9 453 ratings (0,036 %). Vi gjør ikke noe med dem. Ratingene peker på samme film, og `movieId` ligger på hver rating slik at de kan skilles.
- `timestamp` (Unix-tid) gjøres om til `datetime`. Ratings har ingen manglende verdier, duplikater eller verdier utenfor 0,5 til 5, så ingenting fjernes.

2.1.3 Databasemodell og innsetting

Vi bruker to kolleksjoner.

movies (45 433 dokumenter) har ett dokument per film med TMDB-`id` som `_id`. Listefeltene (`genres`, `production_companies`, `production_countries`, `spoken_languages`), samlingen (`belongs_to_collection`), `keywords`, `cast` og `crew` er bygget inn i filmdokumentet (embedding).

Begrunnelse: fem av de ni første spørringene (1, 2, 3, 6 og 8) bruker filmfelt og cast eller crew samtidig, og cast og crew leses alltid sammen med filmen og endres aldri alene. Med egne kolleksjoner hadde nesten hver spørring trengt `$lookup`. Dokumentene er også små nok: den lengste cast-listen har 313 personer og den lengste crew-listen 435, og den største credits-raden er rundt 79 kB som tekst, under 1 % av MongoDBs grense på 16 MB.

ratings (26 024 289 dokumenter) har ett dokument per rating med referanse til filmen.

Begrunnelse: antall ratings per film er ubegrenset (opptil 91 921 for én film), så en innebygd liste ville vokst for hver ny rating og fulgt med hver gang filmen leses, også når ratingene ikke trengs. Oppgave 10 grupperer dessuten per bruker, så alle ratings måtte uansett pakkes ut av filmdokumentene. MovieLens-ID oversettes til TMDB-ID ved innsetting og ikke i spørringene, slik at `tmdbId` peker rett på `movies._id`. Ellers hadde oppgave 10 trengt en ekstra join over 26 millioner rader. `links` er derfor ikke en egen kolleksjon, men brukes bare under innsettingen.

```
movies:  { _id, title, release_date, budget, revenue, runtime,
           vote_average, vote_count, original_language, overview, tagline,
           genres: [{id, name}], production_countries: [{iso_3166_1, name}],
           belongs_to_collection: {id, name, ...} | null,
           keywords: [...], cast: [{id, name, order, gender, ...}],
           crew: [{id, name, job, department, ...}] }
ratings: { userId, movieId, tmdbId | null, rating, timestamp }
```

Innsetting og indekser. Filmene settes inn med `insert_many` i biter på 1 000. Ratings leses i biter på 10 000 med `pandas`, får `tmdbId` lagt på via `links`, og settes inn med `insert_many`. Innsettingskoden er satt opp til å opprette følgende indekser etter innsetting: en tekstindeks over `overview` og `tagline` (oppgave 7), en indeks på `movies.vote_count` og enkeltindekser på `ratings.userId` og `ratings.tmdbId`. Etter innsetting kontrollerte vi antallet: 45 433 filmer og 26 024 289 ratings, som er alle ratings i den fulle `ratings.csv`.

Kontroll av innsettingen

Dokumentantall, datatyper og innebygde lister

Kontroll	Verdi
movies.count_documents({})	45 433
ratings.estimated_document_count()	26 024 289

Eksempelfilm: movies.find_one({"_id": 862})

Felt	Registrert verdi
_id	862
title	Toy Story
release_date	1995-10-30 00:00:00
revenue (USD)	373 554 033
genres	Animation, Comedy, Family
Antall cast-/crew-oppføringer	13 / 106

Utdrag fra en lesekontroll av den eksisterende databasen. Ratings-antallet samsvarer med 26 024 289 rader i råfilen.

Figur 6: Kontroll av dokumentantall og et utdrag av filmdokumentet for Toy Story. Ratings-antallet leses med `estimated_document_count`; det samsvarer med antall rader i den fulle ratings-filen.

2.2 Del 2: Spørringer

Alle funksjonene heter `task_1` til `task_10` og bruker `pprint` til utskrift, slik oppgaveteksten ber om. Resultatfigurene nedenfor viser tabeller laget fra de samme spørringsresultatene for å gjøre dem lettere å lese. Beløp og gjennomsnitt avrundes bare i figurene; beregningene bruker de opprinnelige verdiene.

2.2.1 Oppgave 1: Regissører med høyest medianinntekt

Metode og resultat Spørringen er en aggregasjon på `movies`. Vi bruker `$unwind` på `crew`, beholder `crew.job = Director` og grupperer først på (regissør-ID, film), slik at en regissør som er oppført flere ganger i samme film bare teller én gang. Deretter grupperer vi på regissør-ID og samler antall filmer, gjennomsnittet av `vote_average` og listen med `revenue`. Regissører med under 5 filmer fjernes. MongoDB 7.0.1 tilbyr `$median` med tilnærmet beregning. Vi bruker `statistics.median` i Python for å beregne eksakt median (MongoDB, u.å.-a).

`revenue = 0` behandles som ukjent og er utelatt fra medianen. Antall filmer og snittet av `vote_average` er regnet over alle regissørens filmer, mens medianen bare bygger på filmene med kjent inntekt (`revenue_count`). Regissører uten noen kjent inntekt kan ikke rangeres og er utelatt.

Kyle Balda står øverst med median 1,02 mrd. USD, etterfulgt av David Yates (936 mill.) og Peter Jackson (871 mill.). Listen domineres av regissører bak store franchiser og animasjonsfilmer (*Minions*, *Harry Potter*, *Ringenes herre*, *Avengers*, *Star Wars*, *Ice Age*). Flere medianer bygger på få filmer: Balda har kjent inntekt for 3 av 5 filmer, og Mike Thurmeier og Raman Hui for bare 2 av henholdsvis 6 og 5. Rangeringen er derfor følsom for enkeltfilmer.

queries.py · task_1

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_1(db):
    pipeline = [
        {"$unwind": "$crew"},
        {"$match": {"crew.job": "Director"}},
    # ... (utdrag; hele funksjonen ligger i queries.py)
    for director in results:
        # Inntekt lik 0 behandles som ukjent, slik det er begrunnet i EDA.
        revenues = [value for value in director.pop("revenues")
                    if value is not None and value > 0]
        if revenues:
            director["revenue_count"] = len(revenues)
            director["median_revenue"] = median(revenues)
            ranked.append(director)
    ranked.sort(key=lambda director: (
        -director["median_revenue"], director["_id"]
    ))
    pprint(ranked[:10], sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 7: Kodeutdrag for oppgave 1; hele funksjonen ligger i `queries.py`.

Oppgave 1 · Regissører med høyest medianinntekt

Resultater fra kontrollkjøring

Regissør	Filmer	Kjent inntekt	Medianinntekt (USD)	Snitt vote_average
Kyle Balda	5	3	1 020 063 384	6,620
David Yates	10	6	936 085 968	6,030
Peter Jackson	14	11	871 368 364	7,121
Joss Whedon	5	4	722 136 579	7,400
George Lucas	7	6	712 398 168	6,814
Chris Renaud	5	4	709 485 961	6,700
Mike Thurmeier	6	2	642 911 910	6,667
Brad Bird	6	5	623 722 818	7,150
Francis Lawrence	6	6	619 388 636	6,800
Raman Hui	5	2	592 121 491	6,220

Alle 10 regissører. Medianen bruker bare positive registrerte inntekter.

Figur 8: Resultat fra oppgave 1: topp 10 regissører etter median `revenue`.

2.2.2 Oppgave 2: Skuespillerpar

Metode og resultat Vi grupperer først på (film, skuespiller-ID), slik at en skuespiller med flere roller i samme film bare teller én gang, og samler deretter castet per film. Hver film får to kopier av castlisten (`cast_a` og `cast_b`), begge pakkes ut med `$unwind`, og vi beholder bare par med `cast_a.id < cast_b.id`. Det fjerner par med seg selv og sørger for at (A, B) og (B, A) ikke telles hver for seg. Parene identifiseres med person-ID og ikke navn. Til slutt grupperer vi på paret, teller filmer og tar snittet av `vote_average`, beholder par med minst 3 felles filmer og sorterer synkende på antall filmer. Spørringen bruker `allowDiskUse`, fordi alle par per film blir mange (castlisten kan ha over 300 personer), og tok ca. 134 sekunder.

Det gir 45 650 par. 30 620 av dem har akkurat 3 felles filmer, og antallet faller raskt oppover. Flest felles filmer har Huntz Hall og Leo Gorcey (35 filmer, snitt 3,63), foran Charlie Chaplin og Edna Purviance (33) og Stan Laurel og Oliver Hardy (31). Toppen består av faste komikerduoer fra eldre filmer, men også stemmeskuespillere som Frank Welker, Jeff Bennett og Jim Cummings, som går igjen i animasjonsserier. Snittet for parene fra *Bowery Boys*-filmene er lavt (under 4,5), mens Laurel og Hardy og Chaplin ligger rundt 6,4–6,5.

queries.py · task_2

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_2(db):
    pipeline = [
        # Bare feltene som brukes videre tas med når alle parene lages.
        {"$project": {"cast.id": 1, "cast.name": 1, "vote_average": 1}},
    # ... (utdrag; hele funksjonen ligger i queries.py)
        }},
        {"$project": {
            "cast_a": "$cast",
            "cast_b": "$cast",
            "vote_average": 1
        }},
        {"$unwind": "$cast_a"},
        {"$unwind": "$cast_b"},
        {"$match": {
            "$expr": {"$lt": ["$cast_a.id", "$cast_b.id"]}
    # ... (utdrag; hele funksjonen ligger i queries.py)
        }},
        {"$match": {"movie_count": {"$gte": 3}}},
        {"$sort": {"movie_count": -1, "_id.actor_a": 1, "_id.actor_b": 1}}
    ]
    results = db.movies.aggregate(pipeline, allowDiskUse=True)
    for pair in results:
        pprint(pair, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 9: Kodeutdrag for oppgave 2; hele funksjonen ligger i `queries.py`.

Oppgave 2 · Skuespillerpar

Resultater fra kontrollkjøring

Skuespiller A	Skuespiller B	Felles filmer	Snitt vote_average
Huntz Hall	Leo Gorcey	35	3,631
Charlie Chaplin	Edna Purviance	33	6,461
Oliver Hardy	Stan Laurel	31	6,352
Lou Costello	Bud Abbott	27	6,470
Jeff Bennett	Rob Paulsen	27	6,226
Raymond Burr	Barbara Hale	25	4,980
Grey Griffin	Frank Welker	25	6,708
John Wayne	Paul Fix	24	5,846
Peter Cushing	Christopher Lee	21	5,986
Jim Cummings	Frank Welker	21	6,548
Jim Cummings	Jeff Bennett	21	6,271
Frank Welker	Jeff Bennett	21	6,233
Frank Mayo	Jack Mower	21	5,386
Charlie Chaplin	Henry Bergman	20	6,980
Bernard Gorcey	Leo Gorcey	20	4,550

De første 15 av totalt 45 650 par, sortert etter antall felles filmer.

Figur 10: Resultat fra oppgave 2: de første 15 av totalt 45 650 skuespillerpar, sortert etter antall felles filmer.

2.2.3 Oppgave 3: Skuespillere med størst sjangerbredde

Metode og resultat Vi pakker ut `cast` og grupperer på skuespiller-ID. Per skuespiller samler vi settet av filmer (`$addToSet`, som gir antall krediterte filmer) og settet av sjangerlister. Skuespillere med under 10 filmer fjernes. I Python slås sjangerlistene sammen til ett sett, slik at sjangre som går igjen i flere filmer bare telles én gang. Antall unike sjangre og de fem første sjangrene i alfabetisk rekkefølge tas med som eksempel.

Datasettet har 20 sjangre, og mange skuespillere med minst 10 filmer har alle 20. Topp 10 er derfor en delt førsteplass, og vi har brukt skuespiller-ID som tie-break. Listen viser dermed ti av mange skuespillere med full sjangerbredde og ikke en egentlig rangering: Christopher Lee (148 filmer), Jim Broadbent (78), Michael Madsen (65), Billy Zane (64), Bill Nighy (61), David Carradine og Beau Bridges (60 hver), Stacy Keach (59), Jonathan Pryce (55) og Eddie Izzard (43). Eksempelsjangrene er like for alle ti (*Action, Adventure, Animation, Comedy, Crime*), fordi de er sortert alfabetisk. Filmer uten registrerte sjangre teller med i antall krediterte filmer, men bidrar ikke med nye sjangre. Antall filmer kunne vært en alternativ tie-breaker og ville fremhevet de mest produktive skuespillerne blant dem med lik sjangerbredde; ID brukes her for en entydig rekkefølge uten et ekstra rangeringskriterium.

queries.py · task_3

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_3(db):
    # Topp 10 skuespillere med minst 10 filmer og flest forskjellige sjangre.
    pipeline = [
        {"$unwind": "$cast"},
        {"$group": {
            "_id": "$cast.id",
            "actor": {"$first": "$cast.name"},
            "movies": {"$addToSet": "$_id"},
            "genre_lists": {"$addToSet": "$genres.name"}
        }},
        {"$set": {"movie_count": {"$size": "$movies"}}},
        {"$match": {"movie_count": {"$gte": 10}}},
        {"$project": {"movies": 0}}
    ]
    results = list(db.movies.aggregate(pipeline, allowDiskUse=True))
    for actor in results:
        # Set fjerner sjangre som forekommer i flere av skuespillerens filmer.
        genres = set()
        for genre_list in actor.pop("genre_lists"):
            genres.update(genre_list)
        actor["genre_count"] = len(genres)
        actor["example_genres"] = sorted(genres)[:5]
    results.sort(key=lambda actor: (-actor["genre_count"], actor["_id"]))
    pprint(results[:10], sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 11: Kodeutdrag for oppgave 3; hele funksjonen ligger i `queries.py`.

Oppgave 3 · Skuespillernes sjangerbredde

Resultater fra kontrollkjøring

Skuespiller	Filmer	Unike sjangre
Christopher Lee	148	20
David Carradine	60	20
Michael Madsen	65	20
Jonathan Pryce	55	20
Jim Broadbent	78	20
Stacy Keach	59	20
Eddie Izzard	43	20
Billy Zane	64	20
Beau Bridges	60	20
Bill Nighy	61	20

Alle 10 skuespillere. Ved lik sjangerbredde brukes stigende person-ID.

Fem eksempelsjangre for alle ti: Action, Adventure, Animation, Comedy, Crime.

Figur 12: Resultat fra oppgave 3: skuespillere med flest unike sjangre.

2.2.4 Oppgave 4: Filmsamlinger med størst samlet inntekt

Metode og resultat Vi velger filmer der `belongs_to_collection.name` er en streng (ikke `null`) og grupperer på samlingens ID. Per samling beregnes antall filmer, summen av `revenue`, antall filmer med kjent inntekt, tidligste og seneste `release_date` og listen med `vote_average`. Samlinger med under 3 filmer fjernes. Medianen av `vote_average` beregnes i Python, og datoene formateres som `ÅÅÅÅ-MM-DD`.

`revenue = 0` øker ikke summen. Totalen er derfor summen av registrerte inntekter og kan undervurdere samlingen dersom filmer mangler inntektsopplysninger. Samlinger uten noen kjent inntekt er utelatt. Blant de 10 beste har alle kjent inntekt for alle filmene unntatt *Despicable Me* (4 av 6).

Harry Potter ligger øverst med 7,71 mrd. USD fra 8 filmer (2001–2011, median 7,5), tett fulgt av Star Wars (7,43 mrd., 8 filmer) og James Bond (7,11 mrd.). Bond har klart flest filmer (26), så den har lavest inntekt per film av de tre. De øvrige er *Fast and the Furious* (5,13 mrd.), *Pirates of the Caribbean* (4,52), *Transformers* (4,37), *Despicable Me* (3,69), *Twilight* (3,34), *Ice Age* (3,22) og *Jurassic Park* (3,03). Inntektene er ikke justert for inflasjon, så nyere samlinger favoriseres.

queries.py · task_4

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_4(db):
    # Topp 10 filmsamlinger med minst tre filmer, sortert etter total inntekt.
    pipeline = [
        {"$match": {"belongs_to_collection.name": {"$type": "string"}}},
        {"$group": {
            "_id": "$belongs_to_collection.id",
            "collection": {"$first": "$belongs_to_collection.name"},
            "movie_count": {"$sum": 1},
            "total_revenue": {"$sum": "$revenue"},
            "revenue_count": {"$sum": {"$cond": [{"$gt": ["$revenue", 0]}, 1, 0]}},
            "votes": {"$push": "$vote_average"},
            "earliest_release": {"$min": "$release_date"},
            "latest_release": {"$max": "$release_date"}
        }},
        {"$match": {"movie_count": {"$gte": 3}, "revenue_count": {"$gt": 0}}},
        {"$sort": {"total_revenue": -1, "_id": 1}},
        {"$limit": 10}
    ]
    results = list(db.movies.aggregate(pipeline))
    for collection in results:
        votes = [value for value in collection.pop("votes") if value is not None]
        collection["median_vote_average"] = median(votes) if votes else None
        # Datointervallet bruker bare registrerte datoer; mangler fylles ikke inn.
        for field in ["earliest_release", "latest_release"]:
            if collection[field] is not None:
                collection[field] = collection[field].strftime("%Y-%m-%d")
    pprint(results, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 13: Kodeutdrag for oppgave 4; hele funksjonen ligger i `queries.py`.

Oppgave 4 · Filmsamlinger etter registrert inntekt

Resultater fra kontrollkjøring

Samling	Filmer	Kjent inntekt	Sum inntekt (USD)	Median vote_average	Første / siste utgivelse
Harry Potter	8	8	7 707 367 425	7,50	2001-11-16 2011-07-07
Star Wars	8	8	7 434 494 790	7,45	1977-05-25 2016-12-14
James Bond	26	26	7 106 970 239	6,30	1962-10-04 2015-10-26
The Fast and the Furious	8	8	5 125 098 793	6,65	2001-06-22 2017-04-12
Pirates of the Caribbean	5	5	4 521 576 826	6,90	2003-07-09 2017-05-23
Transformers	5	5	4 366 101 244	6,10	2007-06-27 2017-06-21
Despicable Me	6	4	3 691 070 216	6,90	2010-07-08 2017-06-15
The Twilight	5	5	3 342 107 290	5,80	2008-11-20 2012-11-13
Ice Age	5	5	3 216 708 553	6,50	2002-03-10 2016-06-23
Jurassic Park	4	4	3 031 484 143	6,35	1993-06-11 2015-06-09

Alle 10 samlinger. Totalene gjelder registrerte inntekter og er ikke inflasjonsjustert.

Figur 14: Resultat fra oppgave 4: topp 10 samlinger etter total `revenue`.

2.2.5 Oppgave 5: Spilletid per tiår og primærsjanger

Metode og resultat Vi velger filmer med gyldig `release_date` og minst én sjanger. Tiåret regnes ut fra utgivelsesåret (år minus år modulo 10), og primærsjangeren er første element i `genres`. Vi grupperer på (tiår, primærsjanger), teller filmer og samler `runtime`. Medianen beregnes i Python, og resultatet sorteres på tiår og deretter synkende median.

`runtime = 0` og manglende `runtime` behandles som ukjent og er utelatt fra medianen, men filmene teller fortsatt i `movie_count`. Av 42 965 filmer i resultatet er 1 271 uten kjent spilletid. Én gruppe (2020-tallet, Action) har bare én film uten spilletid, og vises derfor med median `null`.

Det gir 232 grupper. Medianen er 1–1,5 minutter for filmene fra 1870- og 1880-tallet og for de fleste på 1890-tallet (tidlige kortfilmer), og stiger til 60–120 minutter fra 1900-tallet. Fra 1950-tallet ligger de lengste gruppene med mange filmer på 110–120 minutter, typisk War og History (1980-tallet: War, 120 minutter, 32 filmer). På 2010-tallet er medianen for Drama og Action 100 minutter, Comedy 93 og Animation 80. De høyeste verdiene i tidlige tiår bygger ofte på én film (1940-tallet: Science Fiction med 220 minutter), så grupper med få filmer må tolkes med forsiktighet.

queries.py · task_5

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_5(db):
    # Median spilletid og antall filmer per tiår og første registrerte sjanger.
    pipeline = [
        {"$match": {
    # ... (utdrag; hele funksjonen ligger i queries.py)
        }},
        {"$set": {
            "decade": {"$subtract": ["$year", {"$mod": ["$year", 10]}]}
        }},
        {"$group": {
            "_id": {"decade": "$decade", "genre": "$primary_genre"},
            "movie_count": {"$sum": 1},
            "runtimes": {"$push": "$runtime"}
        }}
    ]
    # ... (utdrag; hele funksjonen ligger i queries.py)
    results.sort(key=lambda group: (
        group["_id"]["decade"],
        -(group["median_runtime"] if group["median_runtime"] is not None else -1),
        group["_id"]["genre"]
    ))
    for group in results:
        pprint(group, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 15: Kodeutdrag for oppgave 5; hele funksjonen ligger i `queries.py`.

Oppgave 5 · Spilletid per tiår og primærsjanger

Resultater fra kontrollkjøring

Tiår	Primærsjanger	Filmer	Kjent spilletid	Median (minutter)
1870-tallet	Documentary	2	2	1,0
1880-tallet	Documentary	4	4	1,0
1890-tallet	Fantasy	8	8	1,5
1890-tallet	Action	2	2	1,0
1890-tallet	Comedy	9	9	1,0
1890-tallet	Documentary	27	27	1,0
1890-tallet	Drama	2	2	1,0
1890-tallet	Family	1	1	1,0
1890-tallet	History	1	1	1,0
1890-tallet	Horror	3	3	1,0
1900-tallet	History	1	1	60,0
1900-tallet	Adventure	4	4	15,0
1900-tallet	Drama	7	7	10,0
1900-tallet	Action	2	2	9,0
1900-tallet	Thriller	1	1	8,0

De første 15 av totalt 232 grupper, sortert etter tiår og synkende median spilletid.

Figur 16: Resultat fra oppgave 5: de første 15 av totalt 232 grupper, sortert etter tiår og deretter synkende median spilletid.

2.2.6 Oppgave 6: Kvinneandel blant de fem øverst krediterte

Metode og resultat Spørringen er én aggregasjon på `movies`. Cast sorteres på `order` med `$sortArray`, og de fem første velges med `$slice`. Vi sorterer i stedet for å filtrere på `order < 5`, fordi `order` ikke alltid starter på 0 (rundt 6 000 filmer har første `order` større enn 0). Kvinneandelen er antall med `gender = 1` delt på antall med `gender` 1 eller 2. Ukjent kjønn (`gender = 0`) er altså utelatt fra både teller og nevner. Filmer der ingen av de fem har kjent kjønn, og filmer uten cast eller utgivelsesdato, er utelatt. Etter det gjenstår 39 680 filmer. Andelen per film midles per tiår.

De fleste tiår ligger mellom 0,31 og 0,38. Andelen var lavest på 1970-tallet (0,311) og har steget til 0,376 på 2010-tallet, men den er fortsatt godt under 50 %. De høyeste verdiene (1870-tallet med 1,0 og 2020-tallet med 0,8) bygger på én film hver og sier lite. Det samme gjelder 1890- og 1900-tallet (2 og 12 filmer).

queries.py · task_6

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_6(db):
    """Kvinneandel blant de fem øverst krediterte, per tiår."""
    pipeline = [
        {"$match": {"release_date": {"$ne": None}, "cast.0": {"$exists": True}}},
    # ... (utdrag; hele funksjonen ligger i queries.py)
            "decade": 1,
            "female": {"$size": {"$filter": {"input": "$top5", "cond": {"$eq":
            ["$$this.gender", 1]}}}},
            "known": {"$size": {"$filter": {"input": "$top5", "cond": {"$in":
            ["$$this.gender", [1, 2]]}}}},
        }},
        # Filmer uten noen kjent kjønn blant de fem har ingen andel.
        {"$match": {"known": {"$gt": 0}}},
        {"$group": {
            "_id": "$decade",
            "avg_female_share": {"$avg": {"$divide": ["$female", "$known"]}},
            "movies": {"$sum": 1},
        }},
        {"$sort": {"avg_female_share": -1}},
    ]
    rows = list(db.movies.aggregate(pipeline))
    pprint(rows, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 17: Kodeutdrag for oppgave 6; hele funksjonen ligger i `queries.py`.

Oppgave 6 · Kvinneandel blant de fem øverst krediterte

Resultater fra kontrollkjøring

Tiår	Gjennomsnittlig kvinneandel	Antall filmer
1870-tallet	1,000	1
2020-tallet	0,800	1
1890-tallet	0,500	2
1900-tallet	0,444	12
2010-tallet	0,376	10 543
1930-tallet	0,365	1 249
2000-tallet	0,362	9 647
1940-tallet	0,359	1 435
1990-tallet	0,337	5 179
1910-tallet	0,337	144
1950-tallet	0,336	1 998
1920-tallet	0,331	374
1960-tallet	0,325	2 395
1980-tallet	0,320	3 555
1970-tallet	0,311	3 145

Alle 15 tiår. Andelen beregnes per film blant de fem øverst krediterte med kjent kjønn.

Figur 18: Resultat fra oppgave 6: gjennomsnittlig kvinneandel per tiår.

2.2.7 Oppgave 7: Tekstsøk etter noir og neo-noir

Metode og resultat Vi bruker en tekstindeks over `overview` og `tagline`, opprettet etter innsetting, og søker med `$text` med søkestrengen `noir`. Indeksen deler `neo-noir` i tokenene `neo` og `noir`, så ett søk på `noir` fanger både «noir» og «neo-noir». Vi prøvde først `noir neo-noir"`, men et fraseuttrykk i anførselstegn må også matche, og det ga bare 5 treff uten vote_count-filteret. Resultatet filtreres på `vote_count >= 50`, sorteres på `vote_average` (synkende) og begrenses til 20.

Tekstsøket gir 60 filmer, men bare 5 har minst 50 stemmer, så resultatet har 5 rader. De er *The Bad Sleep Well* (1960, 7,7), *Drunken Angel* (1948, 7,7), *Elevator to the Gallows* (1958, 7,6), *Synchronicity* (2015, 5,7) og *Frank & Lola* (2016, 5,7). Filmer uten `overview` og `tagline` gir ikke treff. Som uavhengig kontroll søkte vi i råfilens `overview` og `tagline` med Python/Pandas (`scripts/verify_noir.py`). Et delstrengsøk etter `noir` gir 7 filmer med minst 50 stemmer. De to ekstra treffene er *The Rules of the Game*, der teksten nevner Jean Renoir, og *Phoenix*, der teksten bruker ordet «Noirish». Med ordgrenser gir regex-kontrollen nøyaktig de samme 5 filmene som tekstindeksen. *Sin City*, *L.A. Confidential*, *Blade Runner*, *Chinatown*, *Drive* og *Pulp Fiction* har ikke delstrengen `noir` i de registrerte tekstfeltene. Resultatet beskriver derfor treff i teksten, ikke alle filmer som filmfaglig kan regnes som noir. Regex og tekstindeks har ulik tokenisering og stemming; vi forventer ikke identisk antall treff før stemmekravet brukes (MongoDB, u.å.-b).

queries.py · task_7

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_7(db):
    """Tekstsøk etter noir / neo-noir."""
    pipeline = [
        # Indeksen deler 'neo-noir' i tokenene 'neo' og 'noir', så 'noir' fanger begge.
        {"$match": {"$text": {"$search": "noir"}, "vote_count": {"$gte": 50}}},
        {"$sort": {"vote_average": -1, "vote_count": -1, "title": 1}},
        {"$limit": 20},
        {"$project": {"_id": 0, "title": 1, "year": {"$year": "$release_date"},
                      "vote_average": 1, "vote_count": 1}},
    ]
    rows = list(db.movies.aggregate(pipeline))
    pprint(rows, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 19: Kodeutdrag for oppgave 7; hele funksjonen ligger i `queries.py`.

Oppgave 7 · Teksttreff på noir

Resultater fra kontrollkjøring

Tittel	År	vote_average	vote_count
The Bad Sleep Well	1960	7,7	57
Drunken Angel	1948	7,7	54
Elevator to the Gallows	1958	7,6	85
Synchronicity	2015	5,7	114
Frank & Lola	2016	5,7	51

Alle 5 filmer med teksttreff og minst 50 stemmer. Ordgrensekontroll i rå CSV gir samme filmer.

Figur 20: Resultat fra oppgave 7.

2.2.8 Oppgave 8: Regissør–skuespiller-samarbeid

Metode og resultat Vi filtrerer på `vote_count >= 100`, henter ut regissørene (`crew.job = Director`) og bruker `$unwind` på regissør og cast. Parene identifiseres med person-ID-ene og ikke navn. Vi grupperer først på (regissør, skuespiller, film), slik at en dobbeltoppføring i cast eller crew ikke teller som to filmer. Deretter grupperer vi på paret og beregner antall filmer og gjennomsnitt av `vote_average` og `revenue`, beholder par med minst 3 filmer og tar topp 20. Par der regissør og skuespiller er samme person i filmen er utelatt. `revenue = 0` behandles som ukjent og er ikke med i gjennomsnittet. Hvis en film mangler revenue, er snittet regnet over de øvrige. Susumu Fujita har ingen kjent revenue og vises derfor uten verdi.

Topplisten domineres av Akira Kurosawa, som har 15 av 20 par, blant annet med Minoru Chiaki (6 filmer, snitt 8,03) og Isao Kimura (4 filmer, 8,05). De øvrige parene er Coppola–Duvall (4 filmer, 8,08), Coppola–Cazale, Tarantino–Keitel, Chaplin–Hank Mann og Miyazaki–Tsunehiko Kamijô. Alle ligger mellom 8,03 og 8,13. Mange par deler akkurat de samme filmene (for eksempel *Seven Samurai*), og det forklarer de identiske snittene.

queries.py · task_8

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_8(db):
    """Regissør-skuespiller-par med minst 3 felles filmer (vote_count >= 100)."""
    pipeline = [
        {"$match": {"vote_count": {"$gte": 100}}},
    # ... (utdrag; hele funksjonen ligger i queries.py)
            "_id": {"d": "$_id.d", "a": "$_id.a"},
            "director": {"$first": "$director"}, "actor": {"$first": "$actor"},
            "films": {"$sum": 1},
            "mean_vote_average": {"$avg": "$vote_average"},
            "mean_revenue": {"$avg": "$revenue"},  # $avg hopper over null (ukjent
            revenue)
        }},
        {"$match": {"films": {"$gte": 3}}},
        {"$sort": {"mean_vote_average": -1, "films": -1, "director": 1, "actor": 1}},
        {"$limit": 20},
    ]
    rows = list(db.movies.aggregate(pipeline, allowDiskUse=True))
    pprint(rows, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «...».

Figur 21: Kodeutdrag for oppgave 8; hele funksjonen ligger i `queries.py`.

Oppgave 8 · Regissør–skuespiller-par, plass 1–10

Resultater fra kontrollkjøring

Regissør	Skuespiller	Filmer	Snitt vote_average	Snittinntekt (USD)
Akira Kurosawa	Eijirô Tôno	3	8,133	271 841
Akira Kurosawa	Haruo Suzuki	3	8,133	271 841
Akira Kurosawa	Minoru Itô	3	8,133	271 841
Charlie Chaplin	Hank Mann	3	8,133	6 506 394
Akira Kurosawa	Atsushi Watanabe	3	8,100	163 540
Akira Kurosawa	Toranosuke Ogawa	3	8,100	163 540
Francis Ford Coppola	John Cazale	3	8,100	99 009 751
Quentin Tarantino	Harvey Keitel	3	8,100	182 573 606
Akira Kurosawa	Daisuke Katô	4	8,075	141 216
Akira Kurosawa	Ichirô Chiba	4	8,075	163 540

Plass 1–10 av topp 20. Bare filmer med minst 100 stemmer inngår; inntekt 0 utelates fra inntektssnittet.

Figur 22: Resultat fra oppgave 8: plass 1–10 av topp 20.

Oppgave 8 · Regissør–skuespiller-par, plass 11–20

Resultater fra kontrollkjøring

Regissør	Skuespiller	Filmer	Snitt vote_average	Snittinntekt (USD)
Akira Kurosawa	Noriko Honma	4	8,075	141 216
Francis Ford Coppola	Robert Duvall	4	8,075	96 622 408
Akira Kurosawa	Haruya Sakamoto	3	8,067	271 841
Akira Kurosawa	Shôichi Hirose	3	8,067	271 841
Akira Kurosawa	Susumu Fujita	3	8,067	Ukjent
Akira Kurosawa	Isao Kimura	4	8,050	163 540
Akira Kurosawa	Minoru Chiaki	6	8,033	141 216
Akira Kurosawa	Masao Shimizu	3	8,033	55 240
Akira Kurosawa	Yûnosuke Itô	3	8,033	55 240
Hayao Miyazaki	Tsunehiko Kamijô	3	8,033	217 150 202

Plass 11–20 av topp 20. Bare filmer med minst 100 stemmer inngår; inntekt 0 utelates fra inntektssnittet.

Figur 23: Resultat fra oppgave 8: plass 11–20 av topp 20.

2.2.9 Oppgave 9: Ikke-engelske filmer med amerikansk tilknytning

Metode og resultat Vi velger filmer der `original_language` ikke er `en` og ikke `null`. En vanlig «ikke lik»-sjekk ville tatt med de 11 filmene uten språk. Amerikansk tilknytning er `production_countries.iso_3166_1 = "US"`. Produksjonsselskapene har bare navn og ID, ikke land. Navnefilteret `^United States` er derfor bare en heuristikk og identifiserer ikke amerikanske selskaper generelt, som Warner Bros. eller Paramount. Det gir ingen ekstra filmer her (452

filmer både med og uten navnefilteret). Resultatet gjelder dermed i praksis filmer med USA som registrert produksjonsland. Delen av spørsmålet om amerikanske produksjonsselskaper kan ikke besvares eksakt fra disse feltene uten en ekstern kobling fra selskaps-ID til land. Dette er en begrensning ved besvarelsen. Vi grupperer på språk, teller, og velger som eksempeltittel filmen med flest stemmer.

Topp 10 er fr (112), es (72), it (56), de (51), ja (31), pt (15), xx (14), nl (12), ru (11) og zh (11). Koden `xx` er ikke et språk. Den brukes i datasettet for filmer uten språk, og eksempelet er *Quest for Fire*. Vi har beholdt den slik den står i dataene.

queries.py · task_9

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_9(db):
    """Ikke-engelske filmer med amerikansk tilknytning."""
    # Selskap mangler landfelt. Navnefilteret under er bare en heuristikk,
    # ikke en fullstendig identifikasjon av amerikanske selskaper.
    us = {"$or": [
        {"production_countries.iso_3166_1": "US"},
        {"production_companies.name": {"$regex": "^United States"}},
    ]}
    pipeline = [
        {"$match": {"original_language": {"$nin": [None, "en"]}, **us}},
        {"$sort": {"vote_count": -1, "title": 1}},  # eksempeltittel = mest stemte film
        {"$group": {"_id": "$original_language", "count": {"$sum": 1}, "example":
        {"$first": "$title"}}},
        {"$sort": {"count": -1, "_id": 1}},
        {"$limit": 10},
    ]
    rows = list(db.movies.aggregate(pipeline))
    pprint(rows, sort_dicts=False)
```

Kodeutdrag fra gjeldende innleveringsfil. Utelatte linjer er markert med «…».

Figur 24: Kodeutdrag for oppgave 9; hele funksjonen ligger i `queries.py`.

Oppgave 9 · Språk blant filmer med USA som produksjonsland

Resultater fra kontrollkjøring

Originalspråk	Antall filmer	Eksempeltittel
fr	112	Leon: The Professional
es	72	Pan's Labyrinth
it	56	The Good, the Bad and the Ugly
de	51	Bagdad Cafe
ja	31	Dragon Ball Z: Resurrection 'F'
pt	15	A Civil Action
xx	14	Quest for Fire
nl	12	Killer Party
ru	11	Brother 2
zh	11	Crouching Tiger, Hidden Dragon

Alle 10 språkgrupper. Selskapsland kan ikke fastslås eksakt fra feltene i datasettet.

Figur 25: Resultat fra oppgave 9.

2.2.10 Oppgave 10: Brukernes sjangerbredde og vurderingsvarians

Metode og resultat Spørringen går over hele `ratings`-samlingen. Vi grupperer på `userId` og beregner antall ratings, `$stdDevPop` (kvadrert gir populasjonsvariansen) og settet av ratede `tmdbId`. Brukere med under 20 ratings fjernes. `tmdbId` ble lagt på hver rating ved innsetting (via `links`). Settet slås opp mot `movies` med `$lookup` på `_id`, og antall ulike sjangernavn telles med `$setUnion`. Ratings uten `tmdbId` (42 707) eller med `tmdbId` som ikke finnes i `movies` teller med i antall og varians, men ikke i sjangertellingen. To topplister lages med `$facet`. Spørringen tok 163,4 sekunder (ca. 2,7 minutter) i kontrollkjøringen.

Datasettet har 20 sjangre, og mange brukere har sett alle. Listen over sjangerbrede brukere er derfor en delt førsteplass, og vi har brukt antall ratings som tie-break (synkende, deretter `userId`). Bruker 45811 har flest ratings (18 276) og alle 20 sjangre. Varians-listen toppes av brukere som bare gir 0,5 og 5,0 (varians 5,0625, som er teoretisk maksimum). Flere av dem har akkurat 20 ratings, så de ligger rett over terskelen.

queries.py · task_10

Samme funksjonsnavn og pprint-format i alle oppgavene

```
def task_10(db):
    """Per bruker: antall ratings, populasjonsvarians og antall ulike sjangre."""
    pipeline = [
        {"$group": {
            "_id": "$userId",
            "n": {"$sum": 1},
            "sd": {"$stdDevPop": "$rating"},
            "movies": {"$addToSet": "$tmdbId"},  # tmdbId er null uten treff i movies
        }},
        {"$match": {"n": {"$gte": 20}}},
        {"$lookup": {"from": "movies", "localField": "movies", "foreignField": "_id",
                     "pipeline": [{"$project": {"_id": 0, "genres.name": 1}}], "as":
                     "rated"}},
        {"$project": {
            "n": 1,
            "variance": {"$pow": ["$sd", 2]},
            "genres": {"$size": {"$reduce": {
                "input": "$rated.genres.name", "initialValue": [],
                "in": {"$setUnion": ["$$value", "$$this"]}}}},
        }},
        {"$facet": {
            "diverse": [{"$sort": {"genres": -1, "n": -1, "_id": 1}}, {"$limit": 10}],
            "variance": [{"$sort": {"variance": -1, "_id": 1}}, {"$limit": 10}],
        }},
    ]
    result = next(db.ratings.aggregate(pipeline, allowDiskUse=True))
    for key, title in [("diverse", "Mest sjangerbrede brukere"), ("variance",
    "Høyest varians")]:
        print(f"\n{title}")
        pprint(result[key], sort_dicts=False)
```

Hele funksjonen fra gjeldende innleveringsfil; blanklinjer er utelatt.

Figur 26: Hele funksjonen for oppgave 10 fra `queries.py`.

Oppgave 10 · mest sjangerbrede brukere

Resultater fra kontrollkjøring

userId	Antall ratings	Populasjonsvarians	Unike sjangre
45811	18 276	0,6439	20
8659	9 279	0,3138	20
270123	7 638	1,1211	20
179792	7 515	0,9000	20
228291	7 410	0,2794	20
243443	6 320	1,6133	20
98415	6 094	0,8486	20
229879	6 024	1,0252	20
98787	5 814	2,0715	20
172224	5 701	0,5334	20

Alle 10 brukere. Minst 20 ratings kreves. Kjøretid for hele spørringen i kontrollkjøringen: 163,4 sekunder.

Figur 27: Oppgave 10: alle 10 brukere med størst sjangerbredde.

Oppgave 10 · brukere med høyest varians

Resultater fra kontrollkjøring

userId	Antall ratings	Populasjonsvarians	Unike sjangre
18703	20	5,0625	13
57842	20	5,0625	14
83102	20	5,0625	15
153882	20	5,0625	15
167241	34	5,0625	14
185889	26	5,0625	14
15395	83	5,0618	14
71009	27	5,0556	13
179759	21	5,0510	13
221535	120	5,0400	19

Alle 10 brukere. Minst 20 ratings kreves. Kjøretid for hele spørringen i kontrollkjøringen: 163,4 sekunder.

Figur 28: Oppgave 10: alle 10 brukere med høyest populasjonsvarians.

3 Diskusjon

3.1 EDA og manglende verdier

Omtrent 80 % av budsjettene og 84 % av inntektene er 0, som vi behandler som ukjent. Vi imputerte ikke, så spørringer på inntekt bygger på et lite utvalg av filmene. I oppgave 1 utelates `revenue = 0` fra medianen; i oppgave 4 øker slike verdier ikke summen. Regissører og samlinger uten noen positiv registrert inntekt utelates. Rangeringene gjelder derfor registrerte inntekter,

og manglende inntekter kan påvirke hvilke filmer som ligger til grunn. Oppgave 8 er et annet eksempel: Susumu Fujita har ingen kjent inntekt og vises uten verdi. Tilsvarende bygger oppgave 7 bare på filmer med `overview` eller `tagline`, og oppgave 6 utelater ukjent kjønn (40 % av cast-oppføringene).

Flere resultater hviler på få filmer eller få stemmer. Tiårene 1870, 1890, 1900 og 2020 i oppgave 6 har 1–12 filmer, og i oppgave 7 er det bare 5 av 60 treff som har nok stemmer. I oppgave 1 bygger flere medianer på bare 2–3 filmer med kjent inntekt, og i oppgave 5 har gruppene fra tidlige tiår få filmer (for eksempel bygger 220 minutter for Science Fiction på 1940-tallet på én film). Oppgave 3 har en delt førsteplass, siden mange skuespillere har alle 20 sjangre, så rekkefølgen på topp 10 avgjøres av tie-break og ikke av ulik sjangerbredde. Brukerne med høyest varians i oppgave 10 har ofte akkurat 20 ratings. Siden fordelingene er så skjeve (figur 4), bruker vi median der oppgavene ber om det, og vi tolker `vote_average` med forsiktighet.

3.2 Datamodell

Det viktigste valget var å bygge cast, crew og keywords inn i filmdokumentet og å legge `tmdbId` direkte på hver rating. Vi vurderte tre alternativer og valgte dem bort:

- **Cast, crew og keywords som egne kolleksjoner.** Forholdet er en-til-en, dataene leses alltid sammen med filmen, og fem av ni spørringer trenger film og cast eller crew samtidig. Det hadde krevd `$lookup` i nesten hver spørring.
- **Ratings innebygd i filmdokumentet.** Den mest ratede filmen har 91 921 ratings. Det ville strengt tatt rommet under 16 MB, men listen vokser uten grense, den leses med filmen også når den ikke trengs, og oppgave 10 grupperer per bruker og måtte uansett pakket ut alle 26 millioner ratings.
- **`links` som egen kolleksjon.** Den trengs bare til å oversette MovieLens-ID til TMDB-ID, og det gjør vi én gang ved innsetting. Da slipper oppgave 10 en ekstra join over 26 millioner rader.

Prisen er at filmdokumentene blir store (den største credits-raden er rundt 79 kB), at spørringer som bare trenger filmfeltene likevel leser mye data, og at `tmdbId` er lagret på hver rating i tillegg til `movieId`. Det siste er redundans, men siden dataene bare lastes inn én gang og ikke oppdateres, vurderer vi risikoen for inkonsistens som lav.

3.3 Datatyper

Tall som var lest inn som tekst (`id`, `budget` og `popularity`) er gjort om til heltall og flyttall, og `release_date` og ratingenes `timestamp` er lagret som `datetime` (UTC), slik at sortering og sammenligning fungerer. Ugyldige verdier gir feilmelding i stedet for å bli `NaN` i det stille. Ukjente verdier lagres som `null`, mens 0 i `budget`, `revenue` og `runtime` er beholdt som den står, og spørringene må selv hoppe over dem.

3.4 Ytelse og begrensninger

Oppgave 1, 3, 4, 5 og 6–9 tok mellom 0,1 og 10 sekunder. Oppgave 2 tok ca. 134 sekunder, fordi den lager alle skuespillerpar per film (castlistene kan ha over 300 personer) og trenger `allowDiskUse`. Ekstra `$lookup` slapp vi her fordi `cast` er bygget inn i filmdokumentet, men prisen er at hele castlisten leses for hver film. Oppgave 10 tok 163,4 sekunder (ca. 2,7 minutter) i kontrollkjøringen. Den grupperer alle 26 millioner ratings og gjør deretter `$lookup` mot filmsettene til hver bruker. Store `$addToSet`-sett, gruppering og filmoppslag kan alle bidra til kjøretiden. `allowDiskUse=True` er allerede aktivert. Uten en måling av de enkelte stegene kan vi ikke fastslå hvor mye tid hvert steg bruker. En indeks på `userId` fjerner ikke behovet for å lese alle ratings.

Et mulig forbedringstiltak er å hente film-ID og sjangre én gang til en Python-dictionary og bruke denne ved sjangeroppslag. Et annet er å bygge sjanger-ID-er inn i hver rating, mot kostnaden av mer lagring og gjentatte opplysninger. Alternativene er ikke ytelsesmålt her, og vi oppgir derfor ingen forventet kjøretid eller garantert hastighetsgevinst.

29 TMDB-filmer er slått sammen fordi flere MovieLens-filmer peker på samme TMDB-id. Det gjelder 0,036 % av ratingene og vurderes som ubetydelig. Koden `xx` i oppgave 9 er beholdt slik den står i dataene.

4 Hva lærte vi

Arbeidet viste hvor mye en grundig EDA styrer resten av oppgaven. Funnene om skjulte nuller, duplikater og de to ID-systemene avgjorde både rensingen og datamodellen. Hadde vi satt inn dataene uten å undersøke dem først, ville medianene i oppgavene med inntekt blitt feil uten at vi la merke til det. Vi lærte også at manglende data ikke alltid skal fylles inn: når rundt 80 % av budsjettene mangler, er det ærligere å utelate dem og si fra.

I MongoDB er modelleringen et valg mellom å bygge inn og å referere, og det rette valget følger av hvordan dataene leses. Cast og crew hører til filmen og bygges inn, mens ratings er for mange og for ujevnt fordelt til det. Vi lærte også at en rask spørring ofte er avgjort allerede ved innsettingen, for eksempel ved å legge `tmdbId` på hver rating. Til slutt ga oppgave 10 en påminnelse om at en aggregering over 26 millioner dokumenter tar tid selv med indekser, og at det lønner seg å teste på små utsnitt først.

5 Tilbakemelding

Det beste med oppgaven er at den bruker et ekte datasett, noe som gjør den mer motiverende enn vanlige øvelser. Man får også kjenne på hele løpet, fra å ta rådata, via å bestemme hvordan de skal ligge i kolleksjoner og få dem inn i databasen, til å hente ut svar med spørringer. Det føles nesten som et lite prosjekt fra arbeidslivet. Samtidig er det et par ting som kunne vært tydeligere. I Task 1 er det vanskelig å vite hvor mye som skal gjøres med dataene før de settes inn, og samtaler med andre studenter tyder på at flere har opplevd det samme. Det står også at det skal leveres en «kort rapport», men ikke hva som menes med kort. Et omtrentlig sideantall hadde gjort det lettere å treffe riktig nivå uten å skrive for mye.

6 Bruk av KI

KI har blitt aktivt brukt som hjelpemiddel i denne oppgaven. Verktøyene som har blitt brukt er Anthropics språkmodell Claude (Anthropic, 2026a), terminalversjonen Claude Code (Anthropic, 2026b) og OpenAIs språkmodeller gjennom ChatGPT (OpenAI, 2026). I tillegg har KI-genererte svar som dukker opp ved vanlige nettsøk blitt benyttet for å svare på enkle kunnskapsspørsmål. Egendefinerte regler for de ulike språkmodellene er opprettet slik at bruken holder seg innenfor NTNUs retningslinjer for KI-bruk. Mer nøyaktig har KI i denne oppgaven blitt brukt til følgende:

- TODO-liste med oversikt over hva som må gjøres for å svare på oppgaven.
- Maler med struktur og hjelpespørsmål som guiding og veiledning.
- Feilsøking av egenskrevet kode og forklaring av kodekonsepter.
- Stikkorddokument for dokumentering av tankeprosess, valg og resultater.
- Språkvask og forslag til struktur og hva rapporten bør inneholde.
- Hjelp med riktig kildeformatering.
- Hjelp med forbedring av figurer og tabeller.

- Sammenslåing av de to EDA-notebookene til én, og kode til figurene i EDA-delen (manglende verdier, fordelinger og ratings per bruker og film).
- Gjennomgang av rapportteksten mot koden for å fange avvik, samt forside og kildehenvisning.
- Formatering av dokumentoppsett i Overleaf med LaTeX.
- Vurdering og tilbakemelding på rapport og kode.
- Kontroll av tekstsøket mot rådata, samkjøring av utskrift med `pprint` og oppdatering av rapportfigurer fra spørringsresultatene.
- Forslag til og endringer i kode (rensing, spørringer, figurer) som gruppen har gått gjennom og tilpasset.

Alt innhold i rapporten og kodebasen er gjennomgått av gruppen og står i tråd med NTNUs retningslinjer: «Artificial intelligence in student papers» (NTNU, u.å.).

Referanser

Anthropic. (2026a). *Claude* [Stor språkmodell]. https://claude.ai

Anthropic. (2026b). *Claude Code* [Programvare]. https://claude.com/product/claude-code

Banik, R. (u.å.). *The Movies Dataset* [Datasett]. Kaggle. https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset

MongoDB. (u.å.-a). *$median*. https://www.mongodb.com/docs/manual/reference/operator/aggregation/median/

MongoDB. (u.å.-b). *Text Index Properties*. https://www.mongodb.com/docs/v7.0/core/indexes/index-types/index-text/text-index-properties/

NTNU. (u.å.). *Artificial intelligence in student papers*. Kunnskapsbasen. https://i.ntnu.no/wiki/-/wiki/English/Artificial+intelligence+in+student+papers

OpenAI. (2026). *ChatGPT* [Stor språkmodell]. https://chatgpt.com